

A UART-Attached Coarse-Grained Reconfigurable Array as a Runtime-Reconfigurable Accelerator

Mattia Fait
Advanced Logic Design
University of Trento
Trento, Italy

Abstract—This report presents the logical design of a coarse-grained reconfigurable array (CGRA) implemented on an FPGA and driven from a host computer over a serial link. The fabric is a 4×4 mesh of 16-bit processing elements (PEs); the host reconfigures the array by loading one configuration word per PE, injects operands at the array edges, advances the datapath a chosen number of cycles and reads back the register file. A compact, checksum-protected UART protocol connects a host-side software stack—a C library and a UNIX-style command-line tool with a small configuration language—to the fabric. We describe the PE datapath, the mesh interconnect, the control finite-state machine, and the mapping of element-wise, reduction and weight-stationary systolic dataflows onto the array under the architectural constraint of a single register per PE with edge-only operand injection.

Index Terms—CGRA, reconfigurable computing, systolic array, FPGA, dataflow, UART protocol.

I. INTRODUCTION

Coarse-grained reconfigurable arrays (CGRAs) occupy the middle ground between the flexibility of software and the efficiency of fixed-function hardware [1]. Rather than reconfiguring at the bit level, as an FPGA does, a CGRA exposes a grid of word-level processing elements (PEs) whose operations and interconnect are set by a compact configuration, allowing the same silicon to be re-purposed at run time for different kernels.

This project implements such a fabric on an FPGA and uses it as a *runtime-reconfigurable accelerator* attached to a host computer over a serial link. The design goals are threefold: (i) a small but expressive fabric whose behaviour is fully controlled, cycle by cycle, from the host; (ii) a robust host/device protocol that can be exercised and verified without the board; and (iii) a UNIX-style software stack that lets a user describe and run accelerated kernels declaratively.

A single architectural decision runs through the whole system and gives the report its narrative spine: each PE owns exactly *one* register and external operands enter the mesh *only* at two edges. We trace the consequences of that decision from the datapath up to the dataflows a user can express and down to the bytes on the wire.

Concretely, this report contributes:

- a fully host-controlled 4×4 CGRA fabric whose every step is gated by a compact, checksum-protected UART protocol (Sections III–IV);
- a mapping of element-wise, reduction and weight-stationary systolic dataflows onto the array, with an explicit argument for *why* the single-register/edge-injection constraint admits some and excludes others (Section V);
- a layered host stack—a thin C library, a byte-accurate device model that serves as a golden reference, and a declarative command-line front end—that runs and is verified end to end without an FPGA (Sections VI, VIII);
- a wire-level cost model that shows the accelerator’s observed speed is bounded by serial round trips rather than by the array (Section VII).

The remainder of this report is organised around the logical model of the system. Section II places the fabric in the CGRA and systolic literature. Section III describes the processing element and the mesh interconnect. Section IV presents the control finite-state machine and the UART protocol. Section V discusses how element-wise, reduction and systolic dataflows map onto the array, including the architectural constraint that shapes those mappings. Section VI outlines the host software stack, Section VII models its wire-level cost, and Section VIII summarises the verification methodology.

II. BACKGROUND AND RELATED WORK

A. The CGRA design space

Coarse-grained reconfigurable arrays trade the bit-level flexibility of an FPGA for word-level processing elements whose operation and interconnect are set by a compact configuration [1]. The design space is broad: PEs range from single ALUs to tightly coupled VLIW datapaths, as in ADRES [2]; interconnects vary from nearest-neighbour meshes to richer crossbars, as in MorphoSys [3]; and configuration may be static for a whole kernel or streamed as a sequence of contexts. More recent fabrics such as Plasticine [5] specialise the PEs and memories around parallel patterns. Across this space the recurring tension is the same: richer PEs and interconnect raise peak throughput but enlarge the configuration and the state that must be managed each cycle.

The fabric described here sits at the minimal end of that spectrum by design. It is a nearest-neighbour mesh of single-

ALU, single-register PEs with a statically loaded, one-word-per-PE configuration. This minimality is not incidental: it is what lets the entire cycle-by-cycle behaviour of the array be exposed to, and driven from, a host over an ordinary serial link, and it is what makes an exact software model of the device tractable.

B. Systolic dataflow

The systolic principle—many identical cells, each doing a little work and passing data to a neighbour on every beat, so that memory bandwidth is amortised over a deep pipeline—dates to Kung [6] and underpins modern matrix engines such as the tensor processing unit [4]. A canonical output-stationary array fuses, in each cell, a multiply with the accumulation of a partial sum arriving from one neighbour while activations flow orthogonally, so each cell holds two independent live values. Section V shows why the single register per PE of this fabric precludes that fusion and selects a *weight-stationary* streaming dataflow instead—the same trade-off that distinguishes weight-from output-stationary accelerators in the literature, here forced by an explicit architectural constraint rather than chosen for energy.

C. Position of this work

This report does not propose a new microarchitecture. Its contribution is a complete, honest, end-to-end *logical* treatment of a deliberately small CGRA: a fully host-controlled fabric, a self-recovering byte-level protocol, a byte-accurate device model that doubles as a golden reference, and a UNIX-style software stack—all connected by a single narrow interface and all verifiable without an FPGA. The value is pedagogical and methodological: it makes visible, in one coherent system, how an architectural constraint propagates all the way up to the dataflows a user can express.

III. FABRIC ARCHITECTURE

The fabric is a $\text{ROWS} \times \text{COLS} = 4 \times 4$ mesh of identical processing elements operating on 16-bit signed words. All PEs share a single clock domain and advance synchronously; a global *step* enable, asserted by the controller, gates every PE register so that the host advances the whole array a deterministic number of cycles per command.

A. Processing element

Figure 1 shows the PE datapath. Each PE has two operand multiplexers that independently select one of seven sources: the registered output of a north, south, east or west neighbour; a 16-bit immediate carried in the configuration word; the PE’s own register (a feedback path); or a constant zero. The selected operands drive an arithmetic-logic unit; its result is latched into the single output register when *step* is asserted.

The ALU implements sixteen operations selected by a 4-bit opcode: a no-op that holds the register; a pass-through; addition, subtraction and multiplication (the low 16 bits of the product); a multiply-accumulate and a simple accumulate, both folding into the register; the bitwise AND/OR /XOR; logical/arithmetic

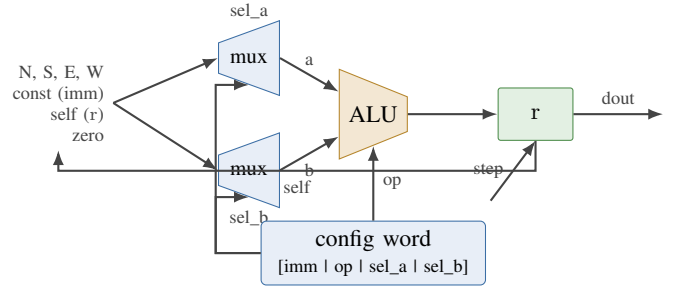


Fig. 1. Processing-element datapath: two operand muxes select from the four neighbours, the immediate, the self register or zero; the ALU result is latched by a single step-gated register.

shifts; signed maximum, minimum and absolute value; and a constant load. Multiplication and multiply-accumulate keep the low half of the result, so arithmetic wraps modulo 2^{16} , consistent with a 16-bit datapath.

A single 32-bit configuration word fully specifies a PE: the low sixteen bits are the immediate, followed by the 4-bit opcode and the two 3-bit operand selects. One such word per PE constitutes a complete array configuration.

B. Mesh interconnect and operand injection

Figure 2 shows the interconnect. Internally, each PE reads the *registered* outputs of its four neighbours, so data advances by at most one hop per step. External operands enter only at two edges: the north input port feeds row 0 (one value per column) and the west input port feeds column 0 (one value per row). The south and east edges read a constant zero. All sixteen PE registers are exposed for read-back.

This edge-only injection, combined with the single register per PE, is the defining architectural constraint of the fabric: interior PEs receive fresh host data only through their neighbours, so any computation that uses the whole array must route operands inward from the edges.

IV. CONTROL AND PROTOCOL

The host drives the fabric through a byte-oriented UART protocol interpreted by a control finite-state machine (FSM). The protocol is deliberately minimal: six single-byte commands, each with a fixed payload and reply, so that the same message stream can be produced and checked by both the hardware and a software model.

A. Command set

Table I lists the commands. Configuration and input transfers carry an 8-bit additive checksum; the device rejects a corrupted transfer with a negative acknowledgement, and appends a checksum to the register read-back. The identification reply reports the array geometry, so the host can adapt at run time rather than assuming fixed dimensions.

B. A transaction on the wire

To make the framing concrete, consider writing the edge inputs west = (1, 0, 0, 0) and north = (5, 0, 0, 0) with the WR

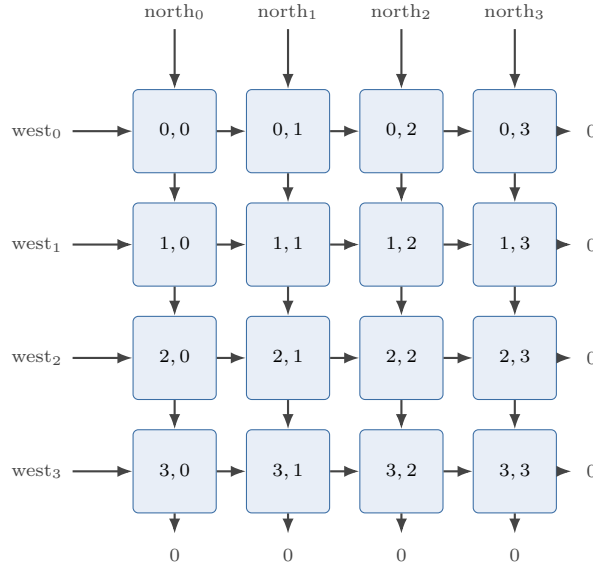


Fig. 2. The 4×4 mesh. Solid arrows are nearest-neighbour register connections; operands are injected only at the north edge (per column) and the west edge (per row), while the south and east edges read zero. This edge-only injection is the central constraint on dataflow mapping (Section V).

TABLE I
UART COMMAND SET (PROTOCOL VERSION 2).

Cmd	Payload	Reply
ID	—	signature, version, ROWS, COLS, width
CFG	16 words + cksum	ACK / NACK
WR	8 words + cksum	ACK / NACK
RUN	step count N	ACK (after N steps)
RD	—	16 registers + cksum
RST	—	ACK (clears registers)

command. The host sends the command byte 0×03 followed by the four west then four north values as 16-bit little-endian words, and finally the additive checksum:

```
03 0100 0000 0000 0000 0500 0000 0000 0000
    06
```

The sixteen payload bytes sum to $1 + 5 = 6$, so the trailing checksum is 0×06 ; the device recomputes the same sum, finds it matches, and replies with a single ACK byte (0×79). A corrupted payload would sum differently and draw a NACK ($0 \times 1F$) instead, on which the host retransmits. Every multi-byte transfer follows this shape—command, fixed payload, one checksum byte, one-byte acknowledgement—which is what lets the same stream be produced and checked identically by the hardware FSM and the software model.

C. Controller state machine

Figure 3 shows the control FSM. From the idle state the first received byte selects a branch. The short transactions (ID, RST and the rejection of an unknown command) reply immediately through a shared transmit handshake. The bulk transfers accumulate their payload, verify (or emit) a checksum

in a dedicated check state, and then reply. The RUN branch asserts *step* once per remaining cycle before acknowledging.

A watchdog timer guards every state that waits for payload bytes: if no byte arrives within a bounded interval the machine aborts back to idle. Together with the checksum, this makes the link self-recovering—a property the host side mirrors by retransmitting rejected transfers and resynchronising after a failed run.

V. DATAFLOW MAPPING

A configuration is only useful if the array can be fed correctly. This section describes how three classes of computation map onto the fabric, and why the architectural constraint of Section III admits some mappings and excludes others.

A. Element-wise and reduction modes

The *diagonal* mapping computes element-wise binary operations. Operand a is injected on the north edge and operand b on the west edge; PEs above the diagonal pass the a values southward and those below pass the b values eastward, so that each diagonal PE receives one (a, b) pair and computes one output. Four elements are produced per transaction, one per diagonal PE.

The *reduction* mapping accumulates a stream into a single PE using the multiply-accumulate operation: at each step the host injects one pair on the north and west ports of PE(0, 0), which folds the product into its register. After the stream is exhausted the register holds the dot product.

B. Weight-stationary systolic matrix-vector

A matrix-vector product $y = Ax$ uses all sixteen PEs. Because interior PEs cannot receive host data directly, the vector x is held *stationary* in the PE immediates while the matrix is *streamed*: row 0 multiplies each incoming matrix

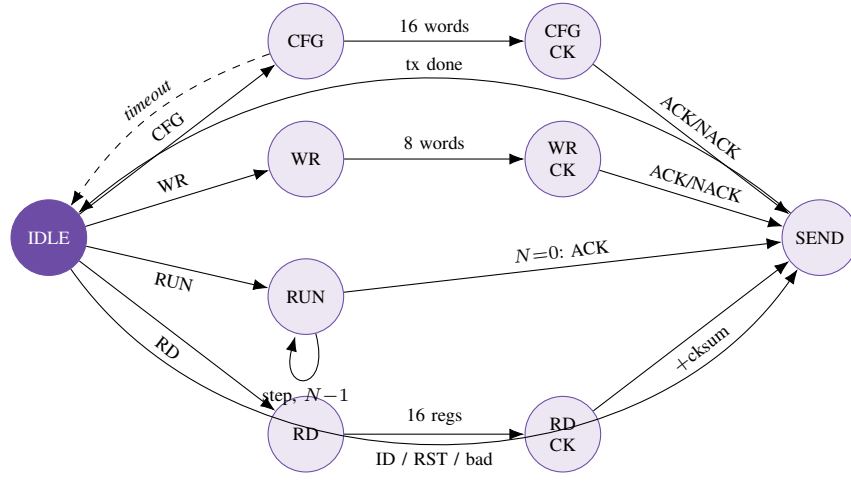


Fig. 3. Control finite-state machine. Payload-bearing commands (CFG, WR, RD) pass through a checksum state before the shared send handshake; RUN pulses step N times. A watchdog (dashed) returns the machine to idle if a payload stalls, so a desynchronised host always recovers.

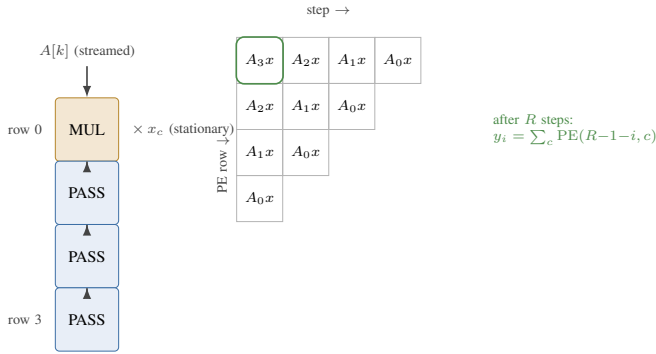


Fig. 4. Weight-stationary systolic matrix-vector product. The matrix streams in one row per step; products march south through the PASS delay line. The skewed diagonal read after R steps gives each output row.

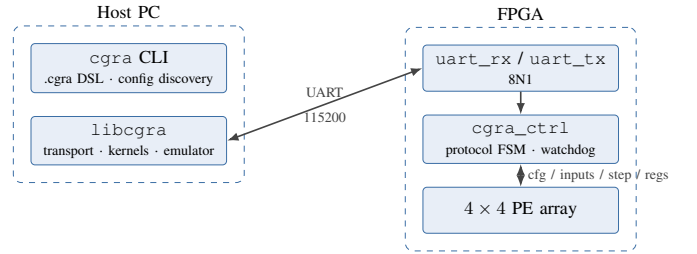


Fig. 5. Layered host/device architecture. The command-line tool compiles a declarative configuration language into per-PE words and drives the library, which reaches the fabric over UART or an in-process emulator.

element by its column weight, and rows 1–3 form a delay line that shifts the partial products southward. Figure 4 shows the resulting space-time skew: after $R = \text{ROWS}$ steps, PE row $R-1-i$ holds the products for output row i , which the host sums. Larger matrices are tiled over 4×4 blocks.

C. Why weight-stationary

A classical output-stationary systolic array (as in a tensor processing unit [4], [6]) fuses, in each PE, a multiply with the addition of a partial sum flowing from a neighbour, while a second stream of activations flows orthogonally. That requires each PE to forward one value *and* retain a distinct accumulator—two independent live values. The present fabric provides a single register per PE, and external data enters only at two edges. Fused spatial accumulation is therefore not realisable here, whereas the weight-stationary streaming dataflow—one operand fixed in the immediates, a single stream marching through PASS cells—is the natural fit. Convolution follows directly, as it is a matrix-vector product with a Toeplitz matrix;

an inclusive prefix scan is expressed similarly on a single row with the running value flowing west to east.

VI. HOST SOFTWARE STACK

The host side (Figure 5) is layered. A C library owns the transport and protocol: it exposes primitives to identify the device, load a configuration, write the edge inputs, advance the array and read the registers, and it implements the checksum, automatic retransmission of rejected transfers, and resynchronisation after a failed run. Crucially, the library also contains an in-process model of the device that speaks the identical byte-level protocol, so the entire stack runs and is verified without an FPGA; the same model can be exposed on a pseudo-terminal to serve external tools.

Above the library, a command-line tool provides high-level, UNIX-style access. A small declarative configuration language describes named *devices*, *modes* (array configurations, expressed either as a high-level pattern or as explicit per-PE placement), *pipelines* that chain modes, and input/output profiles. Configuration is discovered from standard locations and merged by name, following the drop-in convention used by system daemons, so that a user’s library of kernels can be organised across files and overridden selectively. Vectors

TABLE II
WIRE COST PER TRANSACTION (BYTES EACH WAY).

Cmd	TX	RX	Note
ID	1	5	signature + geometry
CFG	66	1	16×4 words + cmd + cksum
WR	18	1	8×2 words + cmd + cksum
RUN	2	1	cmd + step count
RD	1	33	16×2 regs + cksum
RST	1	1	clears registers

flow through the tool via standard input and output, allowing kernels to compose in pipelines with ordinary UNIX utilities.

This separation—a thin, robust library and a declarative front end—keeps the protocol in one place while making the fabric approachable: a user reconfigures the accelerator by naming a mode, not by hand-assembling configuration words.

VII. COST AND PERFORMANCE MODEL

Because the fabric is driven entirely over the serial link, its externally observed performance is set not by the number of PEs but by the number and size of the protocol transactions a kernel issues. This section gives a first-order, purely analytical model of that cost, derived from the protocol of Section IV; it is a wire-level accounting, not a measurement, and deliberately ignores the negligible on-device compute time (a RUN of N steps is N cycles at 100 MHz, dwarfed by the serial exchange).

A. Per-command wire cost

Every transaction is a round trip: the host sends a command and its payload and waits for the reply. Table II lists the byte counts that follow directly from the payload and reply sizes of Table I, for the 4×4 , 16-bit fabric.

B. Per-kernel transaction count

Composing these primitives as the software stack does (Section VI) gives the cost of each kernel. Writing $\lceil \cdot \rceil$ for the ceiling and $T_C = \lceil N/C \rceil$, $T_R = \lceil M/R \rceil$ for the number of column and row tiles, the transaction counts are:

- *Element-wise*, n elements (R lanes per pass): one CFG, then $\lceil n/R \rceil$ rounds of WR+RUN+RD, i.e. $1 + 3\lceil n/R \rceil$ transactions.
- *Reduction* (dot product), n elements: one CFG, one RST, then n rounds of WR+RUN, and one final RD: $3 + 2n$ transactions.
- *Systolic* $M \times N$ matrix–vector: one CFG per column tile and, per tile pair, one RST, R WR+RUN rounds and one RD, i.e. $T_C(1 + 10T_R)$ transactions.

The element-wise cost is dominated by the RD reply (33 bytes) and is amortised only four ways, so throughput is essentially flat in the array size: a wider fabric would inject and read more lanes per round but would not change the per-round round-trip structure.

TABLE III
VERIFICATION SURFACE.

Level	Coverage
RTL (simulation)	PE ALU; full protocol with checksum; systolic matrix–vector
Model (host)	Protocol framing; PE arithmetic; all kernels vs. reference; retransmission
Cross-check	Model is the golden reference for the RTL (shared semantics)

C. Latency model

At 115200 baud, 8N1, each byte occupies ten symbol times, so a byte costs $t_b = 10/115200 \approx 86.8 \mu\text{s}$. Modelling a transaction as the serial time of its two halves, an element-wise round (WR+RUN+RD = 21 TX + 35 RX bytes) takes about $56 t_b \approx 4.9 \text{ ms}$ for four results, i.e. roughly 1.2 ms per element, independent of what the ALU computes. The one-time CFG adds $67 t_b \approx 5.8 \text{ ms}$.

Two conclusions follow, and both point away from the array and toward the link. First, the fixed per-round overhead—a full RD of all sixteen registers regardless of how many lanes were used, plus command and checksum bytes—means small transfers are latency-bound. Second, the natural optimisation is to *batch*: coalescing the write, run and read-back of many chunks into fewer, larger transactions would raise the useful-payload fraction and cut the number of round trips, addressing the true bottleneck identified in Section IX without touching the fabric.

VIII. VERIFICATION

The design is verified at two levels that meet in the middle at the byte-level protocol. On the hardware side, VHDL testbenches driven through the UART interface exercise the processing element in isolation, the full protocol (including checksum rejection and register read-back), and the systolic matrix–vector mapping end to end. On the host side, the software model is used as a golden reference: it re-implements the PE arithmetic and the protocol FSM, so a discrepancy between the two implementations surfaces immediately.

Because the model is byte-accurate, every accelerated kernel is checked against an independent host computation without any hardware: element-wise arithmetic, the dot-product reduction, the tiled systolic matrix–vector product, prefix scans and convolution. The robustness features are exercised too—a fault-injecting variant of the model forces the retransmission path, confirming that a rejected transfer is recovered transparently. The same known-answer suite is reused as a bring-up check on real hardware once the board is programmed.

Table III summarises the verification surface. The bitstream flow itself is scripted for batch synthesis but is out of the scope of this logical report.

IX. CONCLUSION

We have described the logical design of a UART-attached CGRA: a 4×4 mesh of 16-bit processing elements, a

checksum-protected control protocol with a compact finite-state machine, and a layered host stack that reconfigures the fabric declaratively and verifies it end to end without hardware. The recurring theme is a single architectural constraint—one register per PE with edge-only operand injection—which rules out fused spatial accumulation and thereby selects weight-stationary streaming as the natural systolic dataflow for the fabric.

The clearest avenue for future work follows from that same constraint. Adding a second per-PE register (a dedicated partial-sum path) or widening the operand injection would unlock output-stationary systolic mappings and, with them, in-fabric matrix–matrix products and spatial finite-impulse-response filters. On the host side, batching multiple transactions would address the true bottleneck of the real link, which is serial latency rather than the number of processing elements.

REFERENCES

- [1] L. Liu *et al.*, “A Survey of Coarse-Grained Reconfigurable Architecture and Design,” *ACM Comput. Surv.*, vol. 52, no. 6, 2019.
- [2] B. Mei, S. Vernalde, D. Verkest, H. De Man, and R. Lauwereins, “ADRES: An Architecture with Tightly Coupled VLIW Processor and Coarse-Grained Reconfigurable Matrix,” in *Proc. FPL*, 2003, pp. 61–70.
- [3] H. Singh *et al.*, “MorphoSys: An Integrated Reconfigurable System for Data-Parallel and Computation-Intensive Applications,” *IEEE Trans. Comput.*, vol. 49, no. 5, pp. 465–481, 2000.
- [4] N. P. Jouppi *et al.*, “In-Datacenter Performance Analysis of a Tensor Processing Unit,” in *Proc. ISCA*, 2017, pp. 1–12.
- [5] R. Prabhakar *et al.*, “Plasticine: A Reconfigurable Architecture for Parallel Patterns,” in *Proc. ISCA*, 2017, pp. 389–402.
- [6] H. T. Kung, “Why Systolic Architectures?,” *Computer*, vol. 15, no. 1, pp. 37–46, 1982.
- [7] T. Gingold, “GHDL: a VHDL simulator,” <https://github.com/ghdl/ghdl>.